Google L4 DSA — Last-Minute Pattern Cheat Sheet

Visual recognition + core technique + JavaScript mental templates. Built for a fast scan immediately before a 45-minute coding interview.

How to use this: Don't read this like a textbook. Look at the visual → say the pattern out loud → recall the invariant → recall the template → move on.

1. Pattern Recognition Map

If the problem says / feels like…	Think…	Core move
Pair / opposite ends / sorted array	Two Pointers	Move L/R based on condition
Longest / shortest contiguous range	Sliding Window	Expand R, shrink L
Next greater / smaller / unresolved candidates	Monotonic Stack	Pop while invariant breaks
Sorted / monotonic answer space	Binary Search	Discard half
Minimum X / maximum X where feasibility flips	Binary Search on Answer	Binary-search X + can(X)
Level-by-level / minimum edges	BFS	Queue + visited
Explore all reachable nodes	DFS	Recursive or explicit stack
Dependencies / prerequisites	Topological Sort	Indegree or DFS states
Repeated smallest/largest	Heap	Keep best K / next candidate
All combinations / permutations	Backtracking	Choose → recurse → undo
Optimal result from overlapping subproblems	DP	State + transition
Connect components dynamically	Union-Find	Find + union
Prefix lookup / dictionary of prefixes	Trie	Walk characters

2. Two Pointers

L ↓	→	→	→	→	← R
1	2	4	7	9	12

Visual: often sorted array or two ends. Move the pointer that can improve the current relation.

Recognize: sorted array + pair/target; palindrome; in-place compaction; opposite-end optimization.

```
let left = 0, right = nums.length - 1;
while (left < right) {
// evaluate nums[left], nums[right]
if (/* need larger */) left++;
else right--;
}
```

Invariant: every discarded region cannot contain a better/valid answer under the problem's ordering.

3. Sliding Window

L ↓	[	A	B	C	D	]	**→ R**
0	1	2	3	4	5	6	7

Contiguous range. Expand right; when the window becomes invalid, move left until valid again.

Recognize: longest/shortest substring/subarray; at most K; frequency constraints; contiguous range.

```
let left = 0;
for (let right = 0; right < nums.length; right++) {
// add nums[right]
while (/* invalid */) {
// remove nums[left]
left++;
}
// update answer
}
```

4. Binary Search

L ↓	→	→	M ↓	→	→	**R ↓**
2	5	8	12	17	21	30

Ordered candidates. At each step, eliminate one half.

Recognize: sorted data, first/last valid position, rotated sorted array, or a monotonic predicate.

```
let left = 0, right = nums.length - 1;
while (left <= right) {
const mid = Math.floor((left + right) / 2);
if (nums[mid] === target) return mid;
if (nums[mid] < target) left = mid + 1;
else right = mid - 1;
}
```

5. Binary Search on Answer

too small	too small	too small	✓ feasible	✓ feasible	✓ feasible
1	2	3	4	5	6

The values themselves may not be sorted; the FEASIBILITY of an answer is monotonic.

Recognition question: “If X works, will every larger (or smaller) X also work?” If yes, search X.

```
function can(x) {
// return true if x is feasible
}
let lo = minAnswer, hi = maxAnswer;
while (lo < hi) {
const mid = Math.floor((lo + hi) / 2);
if (can(mid)) hi = mid;
else lo = mid + 1;
}
return lo;
```

6. Monotonic Stack

1	3	2	5	4	7
↑	↑	×	↑	×	↑

Keep unresolved candidates in monotonic order. When a new value resolves earlier candidates, pop them.

Recognize: next greater/smaller; previous greater/smaller; temperatures; histogram; nearest boundary.

```
const stack = [];
for (let i = 0; i < nums.length; i++) {
while (stack.length &&
nums[stack[stack.length - 1]] < nums[i]) {
const j = stack.pop();
// nums[i] resolves j
}
stack.push(i);
}
```

Histogram mental model: each bar wants to know how far it can extend left/right before a shorter bar blocks it. The stack maintains increasing bar heights; popping a bar reveals its right boundary and the new stack top gives its left boundary.

7. Largest Rectangle in Histogram — LC 84

2	1	5	6	2	3
■	■	■	■	■	■
		← width →			

When height 5 is popped, its right boundary is the current index; the new stack top determines the left boundary. Area = height × width.

Core recognition: “largest rectangle / nearest smaller boundary” → monotonic increasing stack.

```
const stack = [];
let best = 0;
for (let i = 0; i <= heights.length; i++) {
const h = i === heights.length ? 0 : heights[i];
while (stack.length && heights[stack[stack.length - 1]] > h) {
const height = heights[stack.pop()];
const left = stack.length ? stack[stack.length - 1] : -1;
const width = i - left - 1;
best = Math.max(best, height * width);
}
stack.push(i);
}
```

Interview check: Be able to explain why the width is i - left - 1 and why a sentinel 0 flushes the remaining stack.

8. Trees — DFS / BFS

	A		
B		C	
D	E		**F**

DFS goes deep along a branch; BFS visits one level at a time.

DFS recursion: ask “what should this subtree return to its parent?” This unlocks depth, balance, diameter, path and many subtree problems.

```
function dfs(node) {
if (!node) return 0;
const left = dfs(node.left);
const right = dfs(node.right);
return 1 + Math.max(left, right);
}
```

Iterative DFS:

```
const stack = [root];
while (stack.length) {
const node = stack.pop();
if (!node) continue;
// process node
stack.push(node.right);
stack.push(node.left);
}
```

BFS / level order:

```
const queue = [root];
let head = 0;
while (head < queue.length) {
const node = queue[head++];
// process node
if (node.left) queue.push(node.left);
if (node.right) queue.push(node.right);
}
```

Tree recognition: depth/height → DFS; level-by-level or minimum edges → BFS; BST → exploit ordering.

9. Graphs — BFS / DFS / Topological Sort

A	—	B	—	D
|		|		|
C	—	E		**F**

Think nodes + edges + visited. The same DFS/BFS mechanics apply to adjacency lists and matrix grids.

DFS: reachability, connected components, exploration. **BFS:** shortest path in an unweighted graph / levels. **Grid:** four-direction neighbors + bounds + visited.

```
const queue = [start];
const visited = new Set([start]);
let head = 0;
while (head < queue.length) {
const u = queue[head++];
for (const v of graph.get(u) ?? []) {
if (!visited.has(v)) {
visited.add(v);
queue.push(v);
}
}
}
```

Topological sort recognition: prerequisites, dependencies, ordering. Kahn's algorithm = indegree + queue.

```
const q = [];
for (const node of nodes)
if (indegree[node] === 0) q.push(node);

let head = 0, count = 0;
while (head < q.length) {
const u = q[head++];
count++;
for (const v of graph.get(u) ?? []) {
if (--indegree[v] === 0) q.push(v);
}
```

```
}
// count < n => cycle
```

10. Heaps / Top-K

		10		
	15		20	
18		25		30

A heap gives fast access to the smallest/largest element while retaining only the candidates you need.

Recognize: repeated min/max; Top K; merge sorted streams; scheduling; streaming median.

JS warning: Array.shift() is O(n). For a queue, use a head index. For heaps, know push = bubble up and pop = move root + bubble down.

11. Linked Lists

head	→	A	→	B	→	C	→	null

Pointer manipulation. The most reusable tools are dummy node, prev/current/next, and slow/fast pointers.

High-value set: reverse; cycle; middle; merge two sorted lists; remove Nth from end.

```
let prev = null;
let cur = head;
while (cur) {
const next = cur.next;
cur.next = prev;
prev = cur;
cur = next;
}
return prev;
```

12. Backtracking

start
■■ choose A
■ ■■ choose B → solution
■ ■■ choose C
■■ choose B

Explore a decision tree. Choose → recurse → undo.

```
function backtrack(path) {
if (/* complete */) {
results.push([...path]);
return;
}
for (const choice of choices) {
path.push(choice);
backtrack(path);
path.pop();
}
}
```

13. Dynamic Programming

Recognition: repeated subproblems + optimal/counting decision. Ask: What does dp[i] or dp[i][j] mean? What transition gets me here? What are the base cases?

dp[0]	→	dp[1]	→	dp[2]	→	dp[3]
base		state		state		answer

Start with the smallest state and define exactly what each cell represents.

Last-minute core: Climbing Stairs, House Robber, Maximum Subarray; understand state + transition rather than memorizing code.

14. Union-Find / DSU

A	B	C	D	E
■	■	↑	↑	↑
root A				

Each node points toward a representative. Find follows parents; union connects two components.

Recognize: dynamically joining components, connectivity after many unions, cycle detection in undirected graphs.

Must know: parent array + path compression + union by size/rank. One or two problems is enough for this prep.

15. Trie

root
■■ c ■ a ■ t
■ ■■ r
■■ d ■ o ■ g

Walk one character per level. Excellent for prefix lookup and dictionary-of-words problems.

Recognize: prefix, startsWith, dictionary words, autocomplete-style traversal. Don't over-invest in advanced trie variants.

16. Intervals + Prefix Sums

[1 —— 4]	[3 —— 7]	[9 —— 11]

Sort by start when relationships between intervals matter. Merge when the next start is within the current end.

```
intervals.sort((a, b) => a[0] - b[0]);
for (const [start, end] of intervals) {
if (!out.length || start > out[out.length - 1][1]) {
out.push([start, end]);
} else {
out[out.length - 1][1] =
Math.max(out[out.length - 1][1], end);
}
}
```

Prefix sum recognition: repeated range sums or subarray-sum conditions. Build cumulative totals so a range can be answered from two prefix values.

17. 45-Minute Interview — Mental Checklist

0–5 min	**Clarify**	Restate problem; examples; constraints; edge cases.
5–10 min	**Plan**	Brute force → identify pattern → optimized approach → complexity.
10–30 min	**Code**	Keep talking; use clear variable names; don't over-engineer.
30–38 min	**Test**	Normal case + edge case + empty/singleton/boundary/cycle cases.
38–45 min	**Follow-up**	Complexity, optimization, alternative, or interviewer follow-up.

Final recognition mantra: Pair/sorted → two pointers • contiguous → sliding window • next greater/smaller → monotonic stack • ordered/monotonic answer → binary search • level/shortest unweighted → BFS • explore/reachability → DFS • dependencies → topo sort • repeated min/max → heap • all choices → backtracking • overlapping optimal subproblems → DP • dynamic components → union-find • prefixes → trie.

For LC 84: Think "nearest smaller boundaries" → increasing monotonic stack → pop when current height is smaller → current index is right boundary → new stack top is left boundary → area = height × width.